



RAND

AI Facilitation Group

SESSION 2 ADD-ON · OCTOBER 1 AND 2, 2026

Can you get last Tuesday's version back?

GitHub 101: saving, sharing, and undoing your work. Forty-five minutes, all in the browser, nothing to install.

The rest of the term runs through a repository.

SESSION 3

Scripts need a home

Code you can read is only useful if you can find last week's version of it.

SESSION 4

Agents edit files

Twenty files in a minute. A history turns every change into something you can read and undo.

SESSION 5

Skill docs travel

They're shared as repositories, with a record of every revision.

YOUR THESIS

Replication, mostly done

A clean repository with a README is most of a replication package.

Git keeps the history. GitHub hosts it.

Git

A free program on your own computer. It takes a snapshot of a folder whenever you tell it to, and keeps every snapshot with a one-line reason.

Works offline. Written in 2005 for the Linux source code.

GitHub

A website that keeps a copy of that history online, and adds the parts that involve other people: a page for each project, review of changes, to-do lists, access control.

Owned by Microsoft. GitLab and Bitbucket do the same job.

Four words

01

Repository

A project folder with its full history.

One per project.

02

Commit

A saved version with a one-line reason.

Whenever you reach a state worth returning to.

03

Branch

A parallel draft of the whole project.

To try something without touching the main version.

04

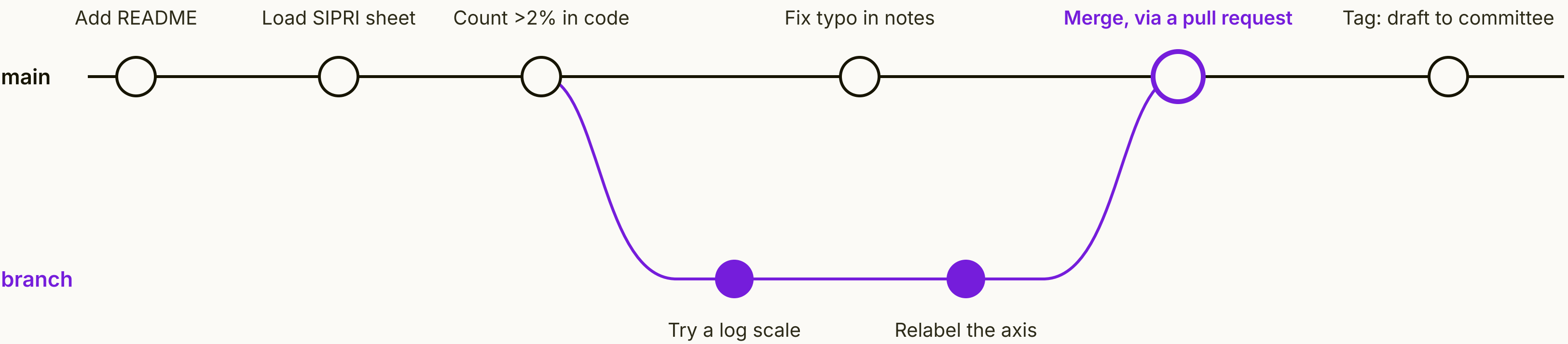
Pull request

A proposal to fold a branch back in, with every change shown.

Before anything joins main, even your own work.

Everything else (forks, clones, Actions, Pages) can wait until you have a reason to want it.

A history is a chain of labeled snapshots.



The experiment lives on its own line until you've read it and decided to keep it.

The tour

-
- 1

The front page. The README renders below the file list. It's the first thing anyone reads, including you in six months.

README.md
-
- 2

The history. Seventy-four commits, each a version you could restore, each with a reason.

"74 commits"
-
- 3

A diff. Red lines removed, green lines added. What you read before you accept anything an agent wrote.

click any commit
-
- 4

What stays off GitHub. The API key and the copyrighted PDFs, listed so Git never tracks them.

.gitignore
-
- 5

The editor trick. Press the period key on any repository and the whole project opens in the browser.

press .
-

Read the diff before you keep it.

count_milex.py	Drop 2019 rows counted twice in the merge
14	df = load_sipri("share_of_gdp.xlsx")
15 -	df = df.merge(regions, on="country")
15 +	df = df.merge(regions, on="country",
16 +	validate="one_to_one")
17	above = df[df["2024"] > 2.0]
18 -	print(len(above))
18 +	print(len(above), "countries")

Red is what left. Green is what arrived.

Whether a person or a model made the change, the diff is the honest account of it. The commit message says why.

Session 3's rule, made structural: if you can't explain the change, don't merge it.

Three things never go up.

.gitignore	never tracked
<pre># keys and settings .env # data and readings stay local data/raw/ literature/ *.pdf</pre>	

.env.example	committed
<pre># Paste your key below, # then rename this file to .env ANTHROPIC_API_KEY=</pre>	

- Keys and passwords**
If one leaks, revoke it first. Deleting the file doesn't erase the history.
- Sensitive or licensed data**
Or RAND work nobody has cleared, even in a private repo.
- Readings you don't own**
Copyrighted PDFs stay on your laptop.

Your first repository, in the browser

- | | | |
|---|---|-------|
| 1 | Create. + menu, New repository. Name it <code>ai-group-notes</code> . Private. Tick "Add a README file." | 2 min |
| 2 | Add. Add file, Create new file, <code>prompts.md</code> . Paste one Session 1 before/after pair. | 3 min |
| 3 | Commit. Write a message that says why. "Add GAO summary prompt pair," not "update." | 1 min |
| 4 | Branch. Edit the README with the pencil, choose "Create a new branch," Propose changes. | 3 min |
| 5 | Review and merge. Create pull request, read the diff, Merge. | 3 min |

Meet a merge conflict here, not in a thesis chapter.

README.md	This branch has conflicts
<pre><<<<<<< fix-intro What this is: notes from the RAND AI group. ===== What this is: my working files, fall 2026. >>>>>>> main</pre>	

Edit the same line of the README on two branches, then try to merge both. Git won't guess which to keep; it marks both versions and asks.

Delete the markers, keep the line you want, and commit. GitHub's "Resolve conflicts" button does this in the browser.

A history makes AI help checkable.

AGENTS · SESSION 4

Every edit becomes a diff

An agent can change twenty files in a minute. With a history, you read each change and can restore any version. Without one, you're trusting its summary.

ATTRIBUTION · SESSION 1

Co-Authored-By

A line at the end of a commit message that names the model. All 74 commits in the demo repository carry one.

FOR STUDENTS

Copilot, free

Verified students get Copilot through GitHub Education. What RAND's approved tool may connect to is a policy question; check first.

Nothing you committed is lost.

Every earlier version is still there. The only work truly at risk is work you haven't committed. So commit small and often, and when something goes wrong, paste the exact error into the model with this:

Git-tutor prompt	takeaway guide
I'm new to Git. Here is exactly what I did and the error I got: [paste]. Explain what happened in plain language, then give me the smallest safe fix. Do not suggest force-push, reset --hard, or deleting the repository.	

BEFORE OCTOBER 15

Homework, and the Playbook

One real thing, three commits

A thesis script, a lit-review table, your prompt pairs. Private repository. Every message says why.

Bring the mistake

The error you hit, and what fixed it. Session 3's scripts go in the same repository.

Start a GitHub section

In the Playbook: the three never-push rules, plus anything that broke today.

The takeaway guide has the click path, the four words, and six links worth an evening: start with GitHub Skills.

ONE MORE FOR THE CHAT

What would you put in a repository first?